

Reviewer Pack — Bipartite Token-Sliding Diameter Lower-Bounds Monotone-KW De...

Entry 299d5162a4f6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 12:48:08 UTC

Bipartite Token-Sliding Diameter Lower-Bounds Monotone-KW Depth

Entry ID: 299d5162a4f6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 12:48:08 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial reconfiguration / token-sliding graphs (Ito-Demaine-Hearn-Etsuko-Uno tradition): the diameter $D_{TS}(B_f)$ of the token-sliding reconfiguration graph on the bipartite minterm/maxterm incidence structure of a monotone Boolean function f , where tokens sit on minterms (1-side) or maxterms (0-side) and a slide moves a token along a ‘shared coordinate’ edge. **Field B** (complexity object): Monotone Karchmer-Wigderson game depth $D_m(f)$ for monotone Boolean functions, equivalently monotone formula depth in the $\{AND, OR\}$ basis

Statement:

For every monotone Boolean function f on n variables, the monotone-KW depth satisfies $D_m(f) \geq \log_2(D_{TS}(B_f) + 1) - 1$, where $D_{TS}(B_f)$ is the diameter (in the connected component containing the canonical all-coordinates token configuration) of the token-sliding graph on the bipartite minterm-maxterm incidence B_f under the move ‘slide token from minterm m to minterm m' iff m and m' differ in exactly one coordinate i with $f(m')=1$, and similarly on the maxterm side’. Equivalently, no monotone formula of depth d can resolve every (minterm, maxterm) pair within fewer than $2^{(d+1)-1}$ token slides, so depth $d < \log_2(D_{TS}+1)-1$ is forbidden for every f . One reconfiguration-diameter witness $D_{TS}(B_f) > 2^{(d+1)}$ refutes any claimed monotone formula of depth $\leq d$ for f .

Rationale (proposer’s reasoning):

A monotone-KW protocol of depth d defines a labeling of (minterm, maxterm) pairs by coordinates that respects f -monotonicity; concatenating coordinate-labels along any reconfiguration path yields a protocol-consistent path on the incidence bipartite graph, and protocol depth bounds the number of distinct decisions reachable by any single token-slide trajectory. Reconfiguration diameters are well-studied combinatorially but, to our knowledge, have never been pulled back through the KW correspondence as a depth lower-bound certificate, despite providing a direct combinatorial obstruction (long sliding paths force deep protocols). The bridge isolates a non-spectral, non-monotone-circuit-counting invariant that is robust under taking sub/superfunctions and sidesteps the natural-proofs barrier because D_{TS} is not closed under random monotone projections.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0d7984f59455c119

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 25 distinct monotone f (n in 3..8) computed deterministically (1 seed per f),
SUPPORT iff every f satisfies $D_m(f) \geq \log_2(D_{TS}(B_f)+1) - 1$ AND OLS regression of $D_m(f)$ on $\log_2(D_{TS}(B_f)+1)$ yields slope ≥ 0.9 and intercept ≥ -1.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.85	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - monotone Karchmer-Wigderson depth token sliding reconfiguration - monotone formula depth minterm maxterm bipartite reconfiguration diameter - combinatorial reconfiguration lower bound monotone circuit complexity

Top relevant hits considered: - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation - [<http://arxiv.org/abs/2504.10671v2>] Token Sliding Reconfiguration on DAGs - [<http://arxiv.org/abs/2002.07444v1>] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [<http://arxiv.org/abs/1904.06696v3>] On bipartite distance-regular Cayley graphs with diameter 3 - [<http://arxiv.org/abs/2302.03695v4>] The Harer-Zagier and Jackson formulas and new results for one-face bipartite maps - [<http://arxiv.org/abs/2410.03416v2>] Asymptotically Optimal Inapproximability of Maxmin k -Cut Reconfiguration - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
import json
from itertools import product, combinations

def generate_monomial(n):
    return tuple(random.choice([0, 1]) for _ in range(n))

def generate_threshold_function(n, k):
```

```

minterms = list(product([0, 1], repeat=n))
selected_minterms = random.sample(minterms, k)
return lambda x: any(x[i] == m[i] for i, m in enumerate(selected_minterms))

def generate_monotone_2cnf(n):
    variables = set(range(n))
    clauses = []
    for _ in range(10): # Generate a few random clauses
        clause = random.sample(variables, 2)
        clauses.append((clause[0], clause[1]))
    return lambda x: all(x[v] or not x[u] for u, v in clauses)

def generate_recursive_majority(n):
    if n == 3:
        return lambda x: (x[0] and x[1]) or (not x[0] and not x[2])
    else:
        subformula = generate_recursive_majority(n - 1)
        return lambda x: subformula(x[:n-1]) and (x[n-1] == 1)

def build_bipartite_graph(f):
    n = len(next(iter(f.keys())))
    minterms = list(product([0, 1], repeat=n))
    maxterms = [tuple(1 - m[i] for i in range(n)) for m in minterms]

    graph = {}
    for m in minterms:
        if m not in graph:
            graph[m] = set()
        for mt in maxterms:
            if mt not in graph:
                graph[mt] = set()
            if sum(m[i] != mt[i] for i in range(n)) == 1 and f(mt) == 1:
                graph[m].add(mt)
                graph[mt].add(m)

    return graph

def bfs_diameter(graph, start):
    queue = [start]
    visited = {start}
    distance = {start: 0}
    max_distance = 0

    while queue:
        current = queue.pop(0)
        for neighbor in graph[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                distance[neighbor] = distance[current] + 1
                max_distance = max(max_distance, distance[neighbor])
                queue.append(neighbor)

    return max_distance

def monotone_kw_depth(f):
    n = len(next(iter(f.keys())))

```

```

memo = {}

def minimax(state, depth=0):
    if state in memo:
        return memo[state]

    if all(f[i] for i in range(n)):
        return depth

    min_val = float('inf')
    for i in range(n):
        new_state = list(state)
        new_state[i] = 1 - new_state[i]
        min_val = min(min_val, minimax(tuple(new_state), depth + 1))

    memo[state] = min_val
    return min_val

return minimax((0,) * n)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 8, 11, 14]
    results = []

    for n in n_values:
        if n == 3:
            f = generate_threshold_function(n, 2)
        elif n == 8:
            f = generate_monotone_2cnf(n)
        elif n == 11:
            f = generate_recursive_majority(3)
        else:
            return {
                "metric_name": "monotone-KW depth",
                "metric_value": None,
                "instances_tested": 0,
                "conjecture_holds": False,
                "counterexample": "mapping_undefined"
            }

        B_f = build_bipartite_graph(f)
        D_TS = bfs_diameter(B_f, (0,) * n)
        D_m = monotone_kw_depth(f)

        results.append({
            "n": n,
            "D_TS": D_TS,
            "D_m": D_m
        })

    if not results:
        return {
            "metric_name": "monotone-KW depth",
            "metric_value": None,

```

```

        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

D_TS_values = [r["D_TS"] for r in results]
D_m_values = [r["D_m"] for r in results]

if all(D_m >= math.log2(D_TS + 1) - 1 for D_m, D_TS in zip(D_m_values, D_TS_values)):
    slope, intercept = linear_regression(D_TS_values, D_m_values)
    if slope >= 0.9 and intercept >= -1.5:
        return {
            "metric_name": "monotone-KW depth",
            "metric_value": None,
            "instances_tested": len(results),
            "conjecture_holds": True,
            "counterexample": ""
        }

for r in results:
    if r["D_m"] < math.log2(r["D_TS"] + 1) - 1:
        return {
            "metric_name": "monotone-KW depth",
            "metric_value": None,
            "instances_tested": len(results),
            "conjecture_holds": False,
            "counterexample": f"Depth {r['D_m']} < log2({r['D_TS']})+1) - 1 for n={r['n']}"
        }

return {
    "metric_name": "monotone-KW depth",
    "metric_value": None,
    "instances_tested": len(results),
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

def linear_regression(x, y):
    n = len(x)
    sum_x = sum(x)
    sum_y = sum(y)
    sum_xy = sum(xi * yi for xi, yi in zip(x, y))
    sum_xx = sum(xi ** 2 for xi in x)

    slope = (n * sum_xy - sum_x * sum_y) / (n * sum_xx - sum_x ** 2)
    intercept = (sum_y - slope * sum_x) / n

    return slope, intercept

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": {"seed": seed, **result}}))

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
{"TRIAL": {"seed": 11, "metric_name": "monotone-KW depth", "metric_value": null, "instances_tested": 0, "counterexample": "mapping_undefined"},  
{"TRIAL": {"seed": 23, "metric_name": "monotone-KW depth", "metric_value": null, "instances_tested": 0, "counterexample": "mapping_undefined"},  
{"TRIAL": {"seed": 37, "metric_name": "monotone-KW depth", "metric_value": null, "instances_tested": 0, "counterexample": "mapping_undefined"},  
{"TRIAL": {"seed": 53, "metric_name": "monotone-KW depth", "metric_value": null, "instances_tested": 0, "counterexample": "mapping_undefined"},  
{"TRIAL": {"seed": 71, "metric_name": "monotone-KW depth", "metric_value": null, "instances_tested": 0, "counterexample": "mapping_undefined"}}
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The stdout_tail shows every trial returned conjecture_holds=false with metric_value=null, instances_tested=0, and counterexample='mapping_undefined' — meaning the test never actually computed monotone-KW depth or D_TS for any instance. This is a construction gap / metric-definition failure: no empirical evidence was produced, so a SUPPORTED verdict (if claimed) would be vacuous. Additionally, the proposed token-sliding move rule is ambiguously specified (the 'similarly on the maxterm side' clause)

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

All 5 trials returned metric_value=null, instances_tested=0, and counterexample='mapping_undefined', meaning the test never computed D_m(f) or D_TS(B_f) for any function. No empirical evidence was produced, and the pre-registered threshold of ≥ 25 functions was not met. | next: Disambiguate the token-sliding move rule (especially the maxterm-side clause) and implement a concrete D_TS computation plus a monotone-KW depth oracle for small n (3..5) before scaling to n=8.

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	24887
2	preregistration	claude_max	opus	0	0	6849
3	novelty	claude_max	opus	0	0	2995
4	novelty	claude_max	opus	0	0	8234
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12079
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17003
7	critic	claude_max	opus	0	0	8650
8	judge	claude_max	opus	0	0	6408

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 87105 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/299d5162a4f6.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/299d5162a4f6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/299d5162a4f6.tar.gz` (if generated)